Αυξάς Θανάσιος ΑΜ: 5401 ΗΥ-240 3η σειρά

Άσκηση 1

α)
```
bool isFull(node*L){
    if(L==NULL)
        return true;
    if((L->LC==NULL && L->RC!=NULL)||(L->LC!=NULL && L->RC==NULL))
        return 0;
    bool l= isFull(L->LC);
    bool r= isFull(L->RC);
    return (l&&r);
}
```

β)
```
bool isKFull(node*L, int K){
    if(L==NULL)
        return true;
    while(L->RS!=NULL && K>0){
        L=L->RS;
        K--;
    }
    if(K>0)
        return 0;
    return isKFull(L->LC,K);
}
```

Задача 2

a)

```
const int MAX;
node* R[1... ∞];

insert(node* root, int K){
    node* prev = NULL;
    node* p = root;
    while (p!=NULL){
        if(K==p->key){
            return; //allready exists
        prev=p;
        if(p->key<K)
            p=p->rc;
        else
            p=p->lc;
    }
    if(prev==NULL)
        exit(1); //empty tree

node* new_node = allocat();
    new_node->lc = new_node->rc = NULL;
    new_node->key = K;
    if(prev->key < K)
        prev->rc = new_node;
    else
        prev->lc = new_node;
}
```

```
tree_insert_delete (node* new_tree, node* node) {
        if (node==NULL)
            return;

        node* left = node->lc;
        node* right = node->rc;

        insert(new_tree, node->key);
        free(node);
        insert_delete(new_tree, left);
        insert_delete(new_tree, right);
}

node* find_successor(node* node){
    while(node != NULL && node->lc != NULL){
        node = node->lc;
    }
    return node;
}

delete (node* Parent, node* Child) {
    node** Parent_Child_Ptr;
    if (Parent->lc == Child)
        Parent_Child_Ptr = &(Parent->lc);
    else
        Parent_Child_Ptr = &(Parent->rc);
    if(Child->lc == NULL && Child->rc == NULL){
        *Parent_Child_Ptr = NULL;
        free(Child);
    }
```

```
    else if(child->rc==NULL){
        *parent_child_ptr=child->lc;
        free(child);
    }
    else if(child->lc==NULL){
        *parent_child_ptr=child->rc;
        free(child);
    }
    else{
        node* successor=find_successor(child->rc);
        child->key=successor->key;
        delete(child,successor);
    }
}

insert_delete_no_left(node* new_tree,node*parent,node*root){
    insert(new_tree,root->data);
    delete(parent,root);
}

insert_delete(node* new_tree,node*parent,node*node,int higher){
    if(node==NULL)
        return;
    insert_delete(new_tree,node->lc);
    node->lc=NULL;
    while(node!=NULL&& node->data<higher)
        insert_delet_no_left(new_tree,parent,node);
```

```
int main() {
    int m=0;
    int higher=-1;
    node* prev=NULL;
    ~~node* p=NULL;~~
    ~~while~~
    node* p=root;
    while(p!=NULL) {
        higher+=MAX;
        if(p->key>higher) {
            prev=p;
            p=p->lc;
        }
        else {
            node* new=allocate();
            new->key=p->key
            R[m]=new;
            insert_delete(R[m],prev,p,higher);
            m++;
            p=root;
        }
    }
```

~~β)~~ ~~foo(node* T1, node* T2){~~

node: tree, cell: list

```
cell* stack1 = T1;
cell* stack2 = T2;
~~cell* stack1->node = T1;~~
~~stack1=~~
~~stack~~
```

β)

```
struct cell {
    node* node;
    cell* next;
};
struct node{
  int data;
  node* lc;
  node* rc;
}
foo(node*T1, node* T2){
    cell* stack1 = alloc(cell);
    stack1 = {NULL, NULL};
    cell* stack2 = alloc(cell);
    stack2 = {NULL, NULL};
    while (T1 != NULL) {
        stack1.push(T1);
        T1 = T1->next;
    }
    while(T2 != NULL) {
        stack2.push(T2);
        T2 = T2->next;
```

```
cell* result = alloc(cell);
result->next = NULL;
cell* end = result;
~~stack1.push(T1);~~
~~stack2.push(T2);~~
```

```
while(T1!=NULL || Stack1!=NULL || T2!=NULL || Stack2!=NULL){
    while(T1){
        Stack1.push(T1);
        T1=T1->lc;
    }
    while(T2){
        Stack2!push(T2);
        T2=T2->lc;
    }
```